

Roteiro de Aula- Selection Sort

1. Definição

Selection Sort, ou ordenação por seleção, é um algoritmo de ordenação por comparação.

A ideia é simples: em cada fase, o algoritmo procura o menor elemento da parte ainda não ordenada e troca esse elemento com a primeira posição da parte não ordenada.

1. A parte à esquerda do vetor fica ordenada aos poucos.
2. A cada fase, uma posição final é definida.
3. O algoritmo usa pouca memória extra: apenas variáveis auxiliares.



Pergunta: Como o Selection Sort decide qual elemento deve ir para a posição atual?

Varrer o vetor procurando o índice do menor valor, aí troca com a posição atual, aí ele procura a partir da posição atual.

2. Antes de programar: simule uma execução

Use o vetor abaixo e complete as fases manualmente.



Vetor inicial: [64, 25, 12, 22, 11]

Fase	Parte analisada	Menor encontrado	Vetor após a troca
i = 0	[64, 25, 12, 22, 11]	11	[11, 25, 12, 22, 64]
i = 1	25, 12, 22, 64	12	[11, 12, 25, 22, 64]
i = 2	5, 22, 64	22	[11, 12, 22, 25, 64]
i = 3	25, 64	25	[11, 12, 22, 25, 64]



Pergunta: Quantas trocas ocorre a cada fase no Selection Sort?

uma troca

3. Complete o código da classe SelectionSort

Crie a classe Selection Sort. Complete as lacunas do código. Repare que a classe é genérica: ela pode ordenar Integer, String ou outros tipos, desde que exista um Comparator.

```
import java.util.Comparator;

public class SelectionSort<T extends Comparable<T>> {
    private long contaComparacoes;
    private long contaTrocas;

    public long getContaComparacoes() {
        return contaComparacoes;
    }

    public long getContaTrocas() {
        return contaTrocas;
    }

    private int indiceMenorValor(T[] v, int posicao, Comparator<T> comparador) {
        int indiceMenor = posicao;

        for (int i = posicao + 1; i < v.length; i++) {
            this.contaComparacoes++;

            if (comparador.compare(v[i], v[indiceMenor]) < 0) {
                indiceMenor = i;
            }
        }

        return indiceMenor;
    }
}
```



Pergunta: Por que o método `indiceMenorValor` retorna o índice do menor valor, e não o valor em si?

Prof. Matheus Franco

```
void troca(T[] v, int i, int j) {
    T temp = v[i];
    v[i] = v[j];
    v[j] = temp;
    this.contaTrocas++;
}

public T[] sort(T[] array, Comparator<T> comparador) {
    int n = array.length;
    this.contaComparacoes = 0;
    this.contaTrocas = 0;

    //for das fases
    for (int i = 0; i < n - 1; i++) {
        int indiceMenor = indiceMenorValor(array, i, comparador);
        troca(array, i, indiceMenor);
    }
    return array;
}

public T[] sort(T[] array) {
    return sort(array, (a, b) -> ((Comparable<T>) a).compareTo(b));
}
```



Pergunta: Em qual linha do código as comparações e trocas estão sendo contadas?

as trocas são nesse método `sort`

e as comparações nesse método `indiceMenorValor`

4. Implementando o main

Agora crie uma classe main para executar Selection Sort e Bubble Sort no mesmo vetor inicial e comparar tempo, trocas e comparações.

Utilize o método para gerar números aleatórios

```
public static Integer[] gerarVetorAleatorio(int tamanho) {  
    Integer[] vetor = new Integer[tamanho];  
    for (int i = 0; i < tamanho; i++) {  
        vetor[i] = (int) (Math.random() * 100);  
    }  
    return vetor;  
}
```

Apresente o código abaixo.



Vetor gerado:[14, 4, 91, 42, 1, 2, 55, 21, 6, 72]

Vetor ordenado:[1, 2, 4, 6, 14, 21, 42, 55, 72, 91]

5. Comparando resultados

Execute o programa e preencha a tabela com os valores exibidos no console.

Algoritmo	N	Comparações	Trocas	Tempo de execução (milis)
Selection Sort	50000	1249975000	49999	3609
Bubble Sort	50000	1249794099	617220214	12758
Selection Sort	150000	11249925000	149999	35187
Bubble Sort	150000	11248747655	5564218491	130322



Pergunta: Qual algoritmo fez menos trocas? Isso confirma a explicação vista em aula?

Selection Sort, o Selection Sort faz apenas uma troca por fase, enquanto o Bubble Sort troca elementos sempre que necessário.

Fique ligado, não se esqueça das premissas abaixo

1. Selection Sort faz tantas comparações quanto o Bubble Sort: para n elementos, são $n(n-1)/2$ comparações.
2. Selection Sort costuma fazer menos trocas que Bubble Sort, porque troca no máximo uma vez por fase.
3. Bubble Sort troca vizinhos fora de ordem e pode fazer muitas trocas quando o vetor está bagunçado.
4. Mesmo que ambos sejam $O(n^2)$, as quantidades de trocas e o tempo medido tendem a ser diferentes.